A Comparative Study on LLM-enabled HPC User Support

Mingkai Zheng
Department of Electrical and Computer Engineering
Rutgers University
New Brunswick, NJ, USA
mz687@scarletmail.rutgers.edu

Fangru Linghu
Department of Electrical and Computer Engineering
Rutgers University
New Brunswick, NJ, USA
fl374@scarletmail.rutgers.edu

Sikan Li
Texas Advanced Computing Center
Texas Advanced Computing Center
Austin, TX, USA
sli@tacc.utexas.edu

Marty Charles Kandes
San Diego Supercomputer Center
San Diego Supercomputer Center
La Jolla, CA, CA, USA
mkandes@sdsc.edu

Niall Gaffney
Texas Advanced Computing Center
Texas Advanced Computing Center
Austin, TX, USA
ngaffney@tacc.utexas.edu

Ian Foster
Department of Computer Science
University of Chicago
Chicago, USA
foster@sc.uchicago.edu

Zhao Zhang
Department of Electrical and Computer Engineering
Rutgers University
New Brunswick, NJ, USA
zhao.zhang@rutgers.edu

Abstract

Providing accurate and efficient user support with the emerging large language model (LLM) techniques for high-performance computing (HPC) centers is extremely challenging. The major obstacle lies in the integration of specialized knowledge that is not available in the base LLMs. Supervised Fine-tuning (SFT), Retrieval Augmented Generation (RAG), and In-Context Learning (ICL), are the common techniques to empower LLMs with domain knowledge. However, there is a lack of comprehensive understanding of their question-answering capabilities in the HPC domain. In this work, we present a comparative study of eight candidate methods by combining the base model (Llama-3.1-8B-Instruct), SFT, RAG, and ICL using a ticket database of 60,000 question-answer pairs from Texas Advanced Computing Center (TACC). We evaluate these methods with five metrics: correctness, coherence, relevance, fluency, and cosine similarity, using both human and LLMs as judges. Our results show that ICL+RAG with the base model achieves the highest correctness in general, and ICL with the base model outperforms others for debugging and testing questions. The results suggest that combining ICL, RAG, and a powerful base model is a promising solution for LLM-enabled user support in computing centers.

CCS Concepts

• **Computing methodologies** → **Natural language generation**;
• **Information systems** → **Information systems applications**.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
PEARC '26, Minneapolis, MN, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2377-3/26/07
https://doi.org/10.1145/3785462.3815796

ACM Reference Format:
Mingkai Zheng, Fangru Linghu, Sikan Li, Marty Charles Kandes, Niall Gaffney, Ian Foster, and Zhao Zhang. 2026. A Comparative Study on LLM-enabled HPC User Support. In *Practice and Experience in Advanced Research Computing (PEARC '26), July 26–30, 2026, Minneapolis, MN, USA.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3785462.3815796

1 Introduction

Modern scientific research relies heavily on large multi-purpose computers such as High-Performance Computing (HPC) and High-Throughput Computing (HTC) systems, GPUs, specialized AI-focused processors, and academic clouds (e.g., Jetstream2 [13]). These systems are used for many purposes: not just to validate hypotheses with numerical simulation [22, 26], but also to formulate hypotheses by analyzing large datasets with statistical and machine learning methods [8, 11] Computing centers such as the National Center for Supercomputing Applications (NCSA), Ohio Supercomputer Center (OSC), San Diego Supercomputer Center (SDSC), Texas Advanced Computing Center (TACC), and campus computing centers support tens of thousands of users with widely varying expertise and experience.

It is challenging to provide high-quality and efficient user support for these computing centers. The HPC experience among users is vastly diverse, from novice users to experienced experts. The software-hardware stacks of modern HPC applications, such as the PyTorch/NCCL/CUDA/GPU stack, are complicated. The question types and update frequencies of relevant data vary significantly. For example, the user data management policy remains unchanged for years while the system availability status is updated every second. The data sources of the answers are heterogeneous, as the relevant information can be in documents, images, or videos.

With the recent breakthroughs in Large Language Models (LLMs) as conversational assistants [20], computing centers have adopted

LLMs to enable HPC assistants that resolve user inquiries [4]. The major challenge of exploiting LLMs as HPC assistants is how to integrate specialized knowledge that is not available in the base LLMs. Several approaches have been explored, including supervised fine-tuning (SFT), retrieval augmented generation (RAG), and in-context learning (ICL). SFT continues to train the pre-trained model with a labeled domain dataset [6, 23]. RAG encodes related data (e.g., documents) in a vector database and then provides related documents to a question as context to LLMs [16]. ICL provides related data sources, such as similar question-answer pairs, about the question to LLMs.

Despite the early deployment of LLM-enabled HPC assistants in computing centers, there is a lack of a comprehensive understanding of the question-answering capability of the candidate methods in the HPC community. Although researchers have compared SFT, RAG, and ICL in other contexts [1, 18, 27]. These studies either covered just a subset of available techniques, targeted closed-ended questions (i.e., answers with limited options, such as "yes" and "no"), or specialized in another domain. For example, Mosbach et al. [18] compared the in-domain and generalization performance of OPT models [31] with SFT and ICL on downstream tasks, including RTE [5] and MNLI [29]. Soudani et al. [27] consider only SFT and RAG. Balaguer et al. [1] consider open-ended questions, which require descriptive and detailed answers, but only in agriculture and with just RAG and SFT. Prior work provides limited insights into our goal of leveraging LLM techniques to enable HPC assistants.

This missing link motivates us to perform a comparative study on SFT, ICL, and RAG to support HPC users. Specifically, we compare the performance of these three techniques and their combinations (e.g., SFT+RAG+ICL) in answering open-ended questions submitted by HPC users. As a case study, we use a ticket database with 60,000 question-answer pairs from TACC as the SFT dataset, ICL examples, and evaluation dataset. To build the RAG database, we leverage the online documents provided by TACC, including general user guides and software documentation. We use five metrics to evaluate the quality of the generated answers: correctness, coherence, relevance, fluency, and the cosine similarity between the generated answer and the response from TACC staff. During the research, we noticed that an efficient and accurate evaluation mechanism is another challenge. We used both human evaluation and LLM-as-a-judge [32] for the evaluation process.

To evaluate the performance of the candidate methods and their combinations, we classify user tickets into four categories: policy inquiries, debugging and testing, technical suggestions, and machine status-related questions. We select `Llama-3.1-8B-Instruct` [12] as the base model and develop the SFT models by fine-tuning it. We compare eight methods by combining various methods for each category. Our results show that ICL+RAG with the base model achieves higher correctness across all categories, while the base model with ICL shows superior correctness for debugging and testing. Surprisingly, the SFT models are less effective in extracting relevant information from external data sources retrieved by RAG and ICL than the base model. Based on the results, we recommend that computing centers consider combining ICL and RAG with powerful base models to build their HPC assistants.

2 Background

In this section, we provide the background on supervised fine-tuning, retrieval augmented generation, and in-context learning.

2.1 Supervised Fine-tuning

LLM training usually starts with unsupervised pre-training on a large, diverse text corpus, where the LLM can learn the basic linguistic structure across different domains, as it has been pointed out by many recent works that LLMs mainly learn knowledge through pre-training [33]. Depending on the size of the pre-training dataset and the model size, pre-training an LLM can take several months on tens of thousands of high-end GPUs. For instance, the Llama 3.1 405B model was pre-trained on a corpus of about 15T tokens on 16K H100 GPUs for 54 days [12].

SFT continues to train the pre-trained model without the original optimizer states on a labeled domain-specific dataset [6, 23]. SFT dataset usually contains sets of carefully curated question-answer pairs, and its number of tokens is much smaller than that of the pre-training corpus. Through SFT, pre-trained models can learn to follow the desired response patterns and format. Although the SFT dataset is usually small, the fine-tuned LLM can perform well and can generalize to unseen tasks. For instance, with an SFT dataset that contains as few as 1,000 question-answer pairs, the fine-tuned Llama 65B model can exhibit performance as strong as DaVinci003 on many tasks [33].

2.2 Retrieval Augmented Generation

Unlike post-training, which continues fine-tuning the pre-trained LLM on customized or private datasets, RAG does not require model training. Instead, RAG builds an external vector database containing useful information, such as software documentation, to answer user questions.

The workflow of RAG is illustrated in Figure 1. The RAG vector database is constructed using an embedding model. Such an embedding model first converts the tokens in a text into a vector of numerical values and then stores them in a database, where the indices are the embedding vectors. Text sources are usually broken into smaller splits, which helps increase the relevance of the retrieval result. When a question is submitted to a RAG pipeline, the embedding model first converts the question into a vector representation. Then, it uses some algorithm (e.g., cosine similarity) to find the most similar vector index in the database and returns the corresponding text. The question, combined with the retrieved information, is then fed as input into the LLM

RAG has many advantages over post-training with private data. First, RAG does not require model training. As the performance of the state-of-the-art model improves, the performance of RAG improves as well. Second, RAG is good at solving questions requiring frequent updates from external sources, such as user account questions. With access to the user account database, RAG can provide the most accurate and up-to-date information to the LLM to help solve the problem better. Finally, with the external data source, RAG helps reduce LLM's hallucination problem.

RAG also has its own limitations. For instance, RAG cannot provide accurate answers when the required information is inaccessible. The hallucination problem in the generated answers cannot be

avoided if the provided information includes fabrication. Moreover, as the external source usually includes irrelevant information, the number of tokens in such an augmented question might exceed the model's maximum context length limit, causing the performance to degrade or even the out-of-memory issue.

2.3 In-Context Learning

In-context learning (ICL), which is also known as few-shot learning, is similar to RAG: it does not require training on the model and relies on external data sources. The difference between them is that ICL's data sources are examples similar to the question. By conditioning the model on the sequence of such demonstrations, the model is expected to answer the question by using the information in the examples and following their patterns. According to OpenAI's work [3], the number of examples is usually proportional to the quality and accuracy of the generated answers.

3 Ticket Database Analysis

In this section, we describe the tickets collected in the TACC database, the data cleaning techniques to extract the main content from tickets, and the categorization strategy.

3.1 Training and Evaluation Dataset

We collect tickets in TACC's database to construct the training dataset for supervised fine-tuning, the data source for the ICL approach, and the evaluation dataset to compare the performance of different approaches. Specifically, we use conversations between TACC staff and users to form a multi-round conversation dataset of more than 60,000 question-answer instances. Then, we use 95% of these instances as the training dataset and used the remaining 5% for evaluation.

It is worth mentioning that a dataset composed of multi-round conversations should not be randomly sampled to form training and evaluation subsets because it is highly likely that a long conversation in the evaluation dataset has a portion of the conversation leaked into the training dataset, causing the evaluation loss to not reflect SFT model's true generalization performance. Therefore, before forming multi-round conversions, we group the conversions of the same ticket and assign it a unique ticket number. Then, we randomly sample the ticket numbers into different partitions. Finally, we construct the multi-round conversion dataset.

We remove the redundant information in the tickets to improve the overall quality of the dataset. The redundant information includes: 1) the metadata generated by the ticketing system, which is mainly auto-generated tags, such as category, queue, subject, requester, date, time, etc., 2) the metadata generated by the email website, including the conversation histories attached at the end of each email, and 3) intermediate emails from TACC: e.g., "`Ticket resolved`", "`Marking as resolved`", and "`A ticket has been created in your Queue.`" As the tickets contain back-and-forth conversations between a user and TACC staff, they are naturally suitable for the dataset, which consists of multi-round question-answering. We exclude images attached to the tickets for two reasons. First, our pre-trained model is not multimodal; it can only handle text as input. Second, after using Optical Character Recognition (OCR) to extract the error messages from the screenshots, we find that the error messages contain thousands of tokens, exceeding the prompt limit for the supervised fine-tuned model.

3.2 Categorization

We categorize the tickets into four categories: technical questions, machine status questions, debugging questions, and policy-related questions. We evaluate the performance of each candidate method for different categories separately. This categorization is performed because questions of different categories may require different sources of information.

The technical questions refer to those that require TACC technical staff's help, including issues with software usage and installation on TACC machines. Some examples are `How to install PyTorch on TACC's LS6`, `How to accelerate file-transferring speed when scp is too slow`, `When will TACC upgrade R to its latest version`, `How to properly set up Multi-Factor Authentication (MFA) on personal device`, and `How to acquire a license for using Matlab on TACC's machine`.

The machine status questions are about network status, the file system, or a particular machine. These questions are raised when users experience a significant slowdown in accessing TACC's file system or cannot submit jobs to SLURM scheduler. To solve these questions, we need the real-time machine status. Although TACC frequently updates machine status and posts maintenance dates on its website, information about the transient machine unavailability is missing.

The debugging questions are those seeking help on runtime errors. These questions are usually attached with a very long error message, but some users prefer to attach a screenshot or the path to the log file to their tickets. However, we did not extract the error messages from the images and error logs because 1) most of the error messages contain thousands of tokens that can easily exceed the context limit of the supervised fine-tuned model, and 2) users' file privacy must be respected.

Most policy-related questions can be solved with reference to TACC's documentation. Some examples are `How to register a TACC account`, `How to reset the password for TACC account`, `How to acquire allocation and machine hours on TACC's Frontera`, `How often does TACC purge files in $SCRATCH`, and `Can TACC help restore accidentally deleted files in $WORK`.

The statistics for the cleaned dataset show that policy-related questions account for the largest share, and machine status questions are the least popular. Additionally, we do not evaluate the performance on machine status questions in this work due to the infeasibility of reproducibility.

4 Experiment Design

We now describe the LLMs used in our experiments, the data collection and processing for RAG and ICL, and the metrics to evaluate the performance of each method.

4.1 Base Model and SFT model

Our experiments are designed to evaluate the performance of different LLM methods on open-ended questions submitted by TACC

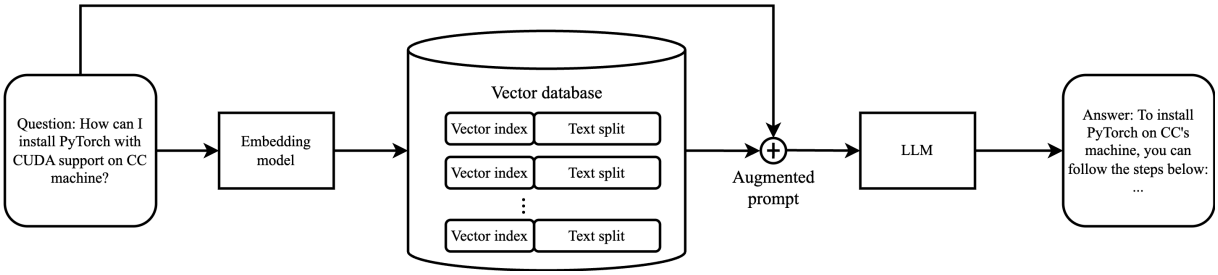


Figure 1: Illustration of RAG and ICL.

users. We consider two models (the base model and the SFT version of the base model). For each method, we consider the model alone, the model with ICL, the model with RAG, and the model with ICL and RAG, for a total of 2×4 = 8 configurations. We use META's `Llama-3.1-8B-Instruct`, one of the most powerful open-source LLMs, as our base model. For SFT models, we fine-tune the base model with cleaned and formatted question-answer pairs of TACC tickets.

4.2 RAG Data Collection and Embedding Model

We collect documentation provided by TACC from three base websites, including `tacc.utexas.edu`, `docs.tacc.utexas.edu`, and `cvw.cac.cornell.edu`. The first URL contains the most up-to-date notice about system status and news. The second one contains the user document for all the machines and software, with examples. The last one contains more commonly used shell scripts for running software on TACC machines. We recursively scrape all the contents on the website under these base URLs. For tables on these websites, directly extracting the text information breaks their structures. Thus, we convert the scraped website to Markdown format to preserve the structural information. When using RAG and ICL, answers generated from context and examples in Markdown format are easier to display, thereby increasing readability.

For those use RAG and ICL, we use `nomic-embed-text-v1.5` [19] as the embedding model, and we used `SKLearnVectorStore` implemented by `Scikit-learn` [21] to build the vector database. We split the text document and question-answer pairs into 400-token chunks to avoid exceeding the maximum context length for LLMs and preserve the information integrity. When generating an answer, the retriever of the vector database returns the top two text splits that are most similar to the question.

4.3 Judge Model and Evaluation Metrics

We evaluate the performance of the candidate methods with five metrics: relevance, coherence, fluency, cosine similarity between the response from TACC staff and the answer generated by LLMs, and correctness. To increase the scalability of accessing relevance, coherence, and fluency, we use LLM-as-a-judge with `DeepSeek-R1-Distill-Qwen-14B` [7] as the judge model and DeepEval framework [14] to quantitatively measure the scores. The evaluation details are as follows.

Relevance checks the relevance between the question and the LLM's generated answer. We directly adopt the built-in Answer Relevancy method of DeepEval to evaluate relevance. The answer is split into several statements. Each statement for its relevance, categorizing them as *yes*, *idk*, or *no*. The final relevance score is calculated as the proportion of relevant statements (*yes* and *idk*) to the total number of statements, emphasizing direct and potential relevance.

Coherence is measured by comparing an LLM's generated answer and the response from TACC. To evaluate coherence, we prompt the model to judge whether the actual and expected outputs are semantically aligned, with an emphasis on meaning rather than superficial similarity in words or sentence structure. The evaluation focuses on three key aspects: 1) Factual information: whether the core facts are consistent across outputs; 2) Content completeness: whether both outputs cover the same key ideas; and 3) Overall coherence: whether the outputs convey the same meaning in a semantically cohesive way. Each aspect is rated as *high*, *medium*, or *low*, with explanations provided for interpretability. The final coherence score is computed as a weighted average over all aspect ratings, where "overall coherence" is given double weight to emphasize its importance. Rating weights are: *high* = 1.0, *medium* = 0.6, and *low* = 0.2. If no verdict is returned, a perfect score of 1.0 is assigned by default.

Fluency evaluates an LLM's language proficiency when answering tickets. To evaluate fluency, we analyze the generated text for linguistic issues such as grammar errors, awkward phrasing, unnatural sentence structure, repetitive expressions, and other disruptions to readability. The evaluation identifies specific problems and categorizes their severity as *high*, *medium*, or *low*, with explanations provided for each issue. Each identified issue contributes to a cumulative penalty score, weighted by severity: *high* = 0.4, *medium* = 0.2, *low* = 0.1. The final fluency score is calculated as 1.0 − penalty, where a perfect fluency (no issues) yields a score of 1.0, and greater penalties proportionally reduce the score.

Cosine similarity: We compute the normalized cosine similarity between the TACC history response and the LLM-generated answer to obtain an unbiased measurement of their similarity. We use the same model, `nomic-embed-text-v1.5`, as in the RAG and ICL approaches.

Correctness: We engage computing center staff to evaluate the correctness of LLM answers. We classify each answer as correct, partially correct, or incorrect. A generated answer is classified as *correct* if it logically follows the operation process of how TACC

staff would address the corresponding question, especially for code debugging questions that require further clarification from the HPC users. For instance, when users who lack experience using HPC systems encounter runtime errors while using TACC machines, they tend to send the error messages only and forget to include the software modules and the path to the code, which are essential for reproducing errors and troubleshooting them. Therefore, in this case, if the LLM asks for more information about the error instead of directly addressing the bug, we still consider it as correct. When the answer contains partial misinformation, we classify it as *partially correct*. If the answer is irrelevant to the question, i.e., only summarizing or repeating the content in the tickets, we mark it as *incorrect*. We assign scores of 1, 0.5, and 0 to correct, partially correct, and incorrect answers, respectively.

5 Experiments and Results

Here, we first describe the experiment setup for fine-tuning and inference, and then we present evaluation results for each method.

5.1 Experiment setup

We apply the same chat template as `Llama-3.1-8B-Instruct` to format both the training and the evaluation dataset. Specifically, the first part of the prompt is the system instruction, defining the role of the LLM. Then, the human and assistant messages are provided alternatively, providing detailed information and questions. When there are consecutive human or assistant messages, we combine them into one message. An example has been provided in Listing 1, where `<|begin_of_text|>`, `<|start_header_id|>`, `<|end_header_id|>`, and `<|eot_id|>` are special tokens defined in the tokenizer, and `system`, `user`, and `assistant` are the role of the corresponding message. If a ticket contains a multi-round conversation between the user and TACC staff, the system message and message content of `user` and `assistant` are placed alternatively.

```
<|begin_of_text|><|start_header_id|>system<|end_header_id
    |>
You are an HPC assistant who answers user tickets.
<|eot_id|><|start_header_id|>system<|end_header_id|>

<|eot_id|><|start_header_id|>user<|end_header_id|>
USER QUESTION
<|eot_id|><|start_header_id|>user<|end_header_id|>

<|eot_id|><|start_header_id|>assistant<|end_header_id|>
```

Listing 1: An example of templated chat

RAG and ICL information are incorporated into the prompt as part of the system messages. For RAG, we insert the retrieved documents immediately after each conversation, following common prompt design practices to enhance the model's use of relevant information. In contrast, the few-shot examples for ICL are placed at the beginning of the conversation, following the system instruction, to clearly demonstrate the desired response style.

We fine-tune `Llama-3.1-8B-Instruct` using DeepSpeed [30] with a maximum context length of 2048 tokens on 64 GH200 GPUs of TACC machine. We choose this length for two main reasons: 1) Only less than 5% of the fine-tuning data contain more than 2048 tokens, and the average number of tokens in these data is more than 60,000, which are either error messages or TACC's internal multi-round conversations across a very long time. Therefore, we consider them as outliers and remove them from the fine-tuning dataset. 2) The context length of 2048 is the maximum number of tokens that can be accommodated on 64 GPUs. We apply many DeepSpeed's unique features to fit the model and data into a compute node's High Bandwidth Memory (HBM) and RAM. First, we use ZeRO stage 3 [24] to partition the AdamW optimizer states, gradients, and model parameters across all GPUs. Second, we leverage DeepSpeed's CPU-offloading [25] to offload the computation of updating model parameters and optimizer states to the CPU. Because of the high computational power provided by the CPU and the high CPU-GPU communication bandwidth (900 GB/s) of GH200, CPU offloading does not introduce significant overhead.

5.2 Results

We apply LLM-as-a-judge with `DeepSeek-R1-Distill-Qwen-14B` as the judge model to evaluate the answers generated by using different schemes quantitatively, including base model (i.e., `Llama-3.1-8B-Instruct`), RAG, ICL, RAG+ICL, SFT (i.e., supervised fine-tuned `Llama-3.1-8B-Instruct`), SFT+ICL, SFT+RAG, and SFT+RAG+ICL. In the following sections, we provide the evaluation scores of five metrics for each category, including technical, debugging, and policy.

5.2.1 Technical questions. From the technical column in Table 1, we see that approaches that use the base model have much higher scores than those using the SFT model. The RAG approach has the highest relevance score, the base model has the highest fluency score, and the ICL approach has the highest scores for coherence and cosine similarity between the generated answer and response from TACC's staff. On the contrary, SFT+RAG+ICL has the lowest scores for coherence and cosine similarity between answers.

These results align with our expectations: ICL and RAG can help improve the answer quality. Although the LLM has seen questions similar to TACC's technical questions and can reason and provide plausible answers without context, using ICL can significantly improve generation quality. However, after using ICL and RAG, the SFT model performs worse. A similar result has been observed in Crystall [28]: when performing SFT with a code dataset on an LLM, its capability of understanding general human instructions and reasoning is degraded.

5.2.2 Debugging questions. The evaluation results of the debugging questions are shown in Table 1 under the debugging column. Similar to the technical questions' results, the base model has the highest relevance and fluency scores, the ICL approach has the highest coherence and cosine similarity between answers, and the SFT+RAG+ICL has the lowest scores for relevance, fluency, and cosine similarity.

These evaluation results further highlight the importance of external data sources for answering TACC tickets. ICL and RAG not only improve the performance of the base model but also help the SFT model generate better answers. However, when combining the SFT model, ICL, and RAG together, the quality of the generated answer degrades, as the SFT model fails to extract helpful information from the external data source.

5.2.3 Policy questions. Most of the policy-related questions can be answered using the documentation on TACC's website, but some questions still require further clarification from TACC staff.

The evaluation scores for each approach on policy-related questions are shown in Table 1 under the policy column, which exhibits similar trends to the evaluation results of the technical and debugging questions: the base model achieves the highest score of relevance and fluency, and the ICL approach has the highest coherence score and the highest cosine similarity between answers. Since answering the policy-related questions is highly dependent on the external data source, we can see that when we use ICL to extract similar questions from the past, the cosine similarity score of the policy questions is higher than that of other categories.

5.2.4 Manually evaluated correctness scores. From the evaluation results above, we can see that the base model usually achieves the highest scores for relevance and fluency, and the answers generated by the ICL approach are the most similar to the response from TACC. However, we still do not have a concrete understanding of the answers' correctness, which is one of the most important metrics when evaluating the quality of the answers.

We invite four computing center staff with extensive experience in answering tickets to manually evaluate the correctness of the answers generated by all approaches above. We classify the answers into three categories: correct, partially correct, and incorrect, with a score of 1, 0.5, and 0, respectively. We evaluate answers generated by 134 questions randomly sampled from the evaluation dataset, consisting of 27 technical, 34 debugging, and 73 policy questions.

The average scores for all categories are shown in Table 1. We can see that in terms of correctness, the ICL+RAG approach has the highest scores for technical and policy-related questions, the ICL approach achieves the highest score for debugging questions, and SFT+RAG has the lowest scores for technical, debugging, and policy questions.

During the manual evaluation process, we observe several interesting phenomena that help explain the correctness scores achieved by different approaches. The SFT+RAG performs the worst because the documentation provided on TACC's website can provide very limited information for solving practical technical, debugging, and policy questions, which explains why the approaches that use ICL have much higher correctness scores. However, the ICL approach with irrelevant examples can greatly influence the generated answers, especially when access to the machine is needed. Taking the issue of SLURM jobs being canceled as an example, there are two possible reasons: 1) the machine has network and file system issues, and 2) the user account is suspended due to improper activities, such as high I/O activities or running computation- or communication-intensive tasks on the login nodes. Without access to the machine to check the file system and the user database to check the user account status, the LLM can only rely on historical examples to generate ungrounded answers.

5.3 Lessons Learned from Experiments

In this subsection, we describe the lessons learned from the experiment results.

5.3.1 Prompt Engineering. The prompt format and prompt engineering play an important role in determining the quality of the generated responses. In early-stage experiments, we utilized the default prompt formatting provided by LangChain to structure model input. This approach offered convenience but lacked the flexibility and token-level controllability. As our work progressed, we moved to manual prompt formatting aligned with the structure outlined in the Llama 3.1 documentation, using model-specific tokens. This resulted in significantly improved generation quality because the model was able to recognize the intended input structure better and focus on the salient information.

5.3.2 Data Privacy. We tried to remove user private information from the question-answer pairs, such as the email address, phone number, and working address, using the Named Entity Recognition (NER) technique with BERT. However, some are still left in our training and evaluation dataset, causing the LLMs to leak such sensitive information to the generated answers.

5.3.3 Supervised Fine-tuning. Supervised fine-tuning with long context length is expensive. With 64 GH200 GPUs and DeepSpeed ZeRO stage 3, we could only fine-tune an 8B-parameter LLM, which is much smaller than most of the powerful open-source LLMs, with a maximum context length of 2048 tokens. As the size of the powerful models continues to increase, the fine-tuning approach for open-ended question-answering tasks becomes increasingly more intractable. Moreover, by comparing the answer qualities above, we have seen that the quality of answers generated by SFT with ICL and RAG is lower than that of other approaches. Although performing supervised fine-tuning on an LLM improves its ability to follow the patterns in history TACC tickets, its performance of extracting useful information from the retrieved external data sources degrades. Additionally, with the augmented input prompt exceeding the SFT model's maximum number of tokens, during the inference process, the special tokens for assistant at the end of the prompt in Listing 1 are truncated by the tokenizer. Without having the correct system and role prompt that the SFT model is trained upon, it will be confused by the instruction, causing its performance to drop. This issue can also be approached by reducing the number of tokens in every text chunk in the vector database, but to preserve the information integrity, we did not try smaller text chunks in our experiment.

6 Related Work

LLMs have been applied in the domain of HPC to address various tasks. For instance, HPC-GPT [9] performs supervised fine-tuning using question-answer instances for two applications: 1) detecting data race in C/C++ and Fortran tasks and 2) managing AI models and datasets for programming language processing and MLPerf tasks. Miyashita et al. [17] propose Hypothetical Command Embedding (HyCE), which extends the existing RAG architecture for HPC-specific data, to simplify the access to HPC data for users unfamiliar with HPC commands and workflows. Jin et al. [15] fine-tuned a Llama model to detect anomalies in computational workflow and compared performance between the fine-tuned LLaMA model and ICL with `GPT2-127M`, `Mistral-7B-v0.1`, and `LLama2-7B`.

Table 1: Evaluation metrics for Policy, Debugging, and Technical questions (Rel.=relevance, Flu.=fluency, Coh.=coherence, CosSim=cosine similarity). Manual column is the manually evaluated correctness score. (Boldface and underline indicate the highest and lowest score in each column, respectively.)

Method	Technical					Debugging					Policy				
	Rel.	Flu.	Coh.	CosSim	Manual	Rel.	Flu.	Coh.	CosSim	Manual	Rel.	Flu.	Coh.	CosSim	Manual
Base	0.909	**0.730**	0.272	0.652	0.591	**0.921**	**0.735**	0.283	0.620	0.591	**0.929**	**0.745**	0.275	0.657	0.597
ICL	0.904	0.706	**0.364**	**0.694**	0.579	0.913	0.706	**0.382**	**0.663**	**0.700**	0.900	0.728	**0.374**	**0.704**	0.583
RAG	**0.910**	0.674	0.296	0.653	0.433	0.908	0.672	0.303	0.622	0.563	0.888	0.691	0.264	0.658	0.550
RAG+ICL	0.888	0.685	0.353	0.672	**0.605**	0.915	0.678	0.356	0.641	0.600	0.895	0.695	0.322	0.681	**0.643**
SFT	0.633	0.639	0.260	0.613	0.357	0.614	0.633	0.261	0.596	0.457	0.657	0.630	0.295	0.663	0.397
SFT+ICL	0.687	0.603	0.351	0.664	0.425	0.649	0.628	0.359	0.634	0.407	0.636	0.611	0.317	0.665	0.386
SFT+RAG	0.530	0.637	0.239	0.574	0.217	0.542	0.655	0.231	0.552	0.342	0.569	0.651	0.243	0.595	0.331
SFT+RAG+ICL	0.534	0.620	0.239	0.562	0.385	0.457	0.627	0.256	0.542	0.444	0.533	0.593	0.240	0.576	0.392

The LM4HPC framework [10] applied LLMs to analyze and optimize HPC software for research and development purposes. Biggs et al. [2] adopted an LLM with a customized vector database and RAG to help HPC staff answer tickets submitted by users of Idaho National Laboratory (INL). However, they did not compare the answer quality of RAG with other existing techniques.

Many existing works have compared the performance of SFT, ICL, and RAG. For example, Mosbach et al. [18] compared the in-domain and out-of-domain performance of ICL and supervised fine-tuned OPT model using RTE and MNLI dataset. Soudani et al. [27] compared RAG, SFT, and SFT+RAG on the less popular dataset. Balaguer et al. [1] compared the performance of SFT and RAG on open-ended agriculture related questions. However, these works are limited to comparing some techniques of SFT, RAG, and ICL and do not compare them within HPC context.

7 Conclusion and Future Work

In this work, we design and evaluate SFT, ICL, and RAG, as well as their possible combinations for the application of answering tickets from HPC centers. We use five metrics to evaluate the quality of the generated answers: relevance, coherence, fluency, cosine similarity between the generation and response from TACC, and correctness. We use LLM-as-a-judge to automate the evaluation of relevance, coherence, fluency, and cosine similarity, and we manually evaluate correctness. Through extensive experiments and careful comparison studies, we observe that ICL+RAG performs better than other approaches in terms of correctness. Although the base model without ICL or RAG achieves the highest scores for relevance and fluency, its correctness is much lower than ICL+RAG. The supervised fine-tuning model fails to extract important information from the external data sources retrieved by RAG and ICL. Through the manual evaluation process, the groundedness and effectiveness of the retrieved information can significantly affect the accuracy of the generated answers. To further improve the correctness of LLMs for answering user tickets, we will combine an AI agent with access to the system and the user database to provide real-time information for LLM generation in future work.

Acknowledgments

This research is supported by NSF under OAC-2411294, OAC-2411296, and OAC-2411297. The authors acknowledge the Texas Advanced Computing Center (TACC) at The University of Texas at Austin for providing computational resources that have contributed to the research results reported within this paper. URL: http://www.tacc.utexas.edu.

References

[1] Angels Balaguer, Vinamra Benara, Renato Luiz de Freitas Cunha, Roberto de M. Estevão Filho, Todd Hendry, Daniel Holstein, Jennifer Marsman, Nick Mecklenburg, Sara Malvar, Leonardo O. Nunes, Rafael Padilha, Morris Sharp, Bruno Silva, Swati Sharma, Vijay Aski, and Ranveer Chandra. 2024. RAG vs Fine-tuning: Pipelines, Tradeoffs, and a Case Study on Agriculture. arXiv:2401.08406 [cs.CL] https://arxiv.org/abs/2401.08406

[2] Brandon S Biggs and Kaylee Dalton. 2024. *Supplementing HPC Support with a Science Gateway AI Assistant.* Technical Report. Idaho National Laboratory (INL), Idaho Falls, ID (United States).

[3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. arXiv:2005.14165 [cs.CL] https://arxiv.org/abs/2005.14165

[4] Franck Cappello, Sandeep Madireddy, Robert Underwood, Neil Getty, Nicholas Lee-Ping Chia, Nesar Ramachandra, Josh Nguyen, Murat Keceli, Tanwi Mallick, Zilinghan Li, et al. 2025. EAIRA: Establishing a Methodology for Evaluating AI Models as Scientific Research Assistants. *arXiv preprint arXiv:2502.20309* (2025).

[5] Ido Dagan, Oren Glickman, and Bernardo Magnini. 2006. The PASCAL Recognising Textual Entailment Challenge. In *Machine Learning Challenges. Evaluating Predictive Uncertainty, Visual Object Classification, and Recognising Tectual Entailment*, Joaquin Quiñonero-Candela, Ido Dagan, Bernardo Magnini, and Florence d'Alché Buc (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 177–190.

[6] Andrew M. Dai and Quoc V. Le. 2015. Semi-supervised Sequence Learning. arXiv:1511.01432 [cs.LG] https://arxiv.org/abs/1511.01432

[7] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL] https://arxiv.org/abs/2501.12948

[8] Jonas Degrave, Federico Felici, Jonas Buchli, Michael Neunert, Brendan Tracey, Francesco Carpanese, Timo Ewalds, Roland Hafner, Abbas Abdolmaleki, Diego de Las Casas, Craig Donner, Leslie Fritz, Cristian Galperti, Andrea Huber, James Keeling, Maria Tsimpoukelli, Jackie Kay, Antoine Merle, Jean-Marc Moret, Seb Noury, Federico Pesamosca, David Pfau, Olivier Sauter, Cristian Sommariva, Stefano Coda, Basil Duval, Ambrogio Fasoli, Pushmeet Kohli, Koray Kavukcuoglu, Demis Hassabis, and Martin Riedmiller. 2022. Magnetic control of tokamak plasmas through deep reinforcement learning. *Nature* 602, 7897 (2022), 414–419.

[9] Xianzhong Ding, Le Chen, Murali Emani, Chunhua Liao, Pei-Hung Lin, Tristan Vanderbruggen, Zhen Xie, Alberto Cerpa, and Wan Du. 2023. HPC-GPT: Integrating large language model for high-performance computing. In *Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*. 951–960.

[10] Murali Emani and Bronis de Supinski. 2023. LM4HPC: Towards Effective Language Model Application in High-Performance Computing. In *OpenMP: Advanced Task-Based, Device and Compiler Programming: 19th International Workshop on OpenMP, IWOMP 2023, Bristol, UK, September 13–15, 2023, Proceedings*, Vol. 14114. Springer Nature, 18.

[11] Linjing Fang, Fred Monroe, Sammy Weiser Novak, Lyndsey Kirk, Cara R. Schiavon, Seungyoon B. Yu, Tong Zhang, Melissa Wu, Kyle Kastner, Yoshiyuki Kubota,

Zhao Zhang, Gulcin Pekkurnaz, John Mendenhall, Kristen Harris, Jeremy Howard, and Uri Manor. 2021. Deep learning-based point-scanning super-resolution imaging. *Nature Methods* 18, 4 (2021), 406–416.

[12] Aaron Grattafiori and *et al.* 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[13] David Y Hancock, Jeremy Fischer, John Michael Lowe, Winona Snapp-Childs, Marlon Pierce, Suresh Marru, J Eric Coulter, Matthew Vaughn, Brian Beck, Nirav Merchant, et al. 2021. Jetstream2: Accelerating cloud computing via Jetstream. In *Practice and Experience in Advanced Research Computing 2021: Evolution Across All Dimensions*. 1–8.

[14] Jeffrey Ip and Kritin Vongthongsri. 2025. *deepeval*. https://github.com/confident-ai/deepeval

[15] Hongwei Jin, George Papadimitriou, Krishnan Raghavan, Pawel Zuk, Prasanna Balaprakash, Cong Wang, Anirban Mandal, and Ewa Deelman. 2024. Large Language Models for Anomaly Detection in Computational Workflows: from Supervised Fine-Tuning to In-Context Learning. arXiv:2407.17545 [cs.SE] https://arxiv.org/abs/2407.17545

[16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401 [cs.CL]

[17] Yusuke Miyashita, Patrick Kin Man Tung, and Johan Barthélemy. 2024. LLM as HPC Expert: Extending RAG Architecture for HPC Data. *arXiv preprint arXiv:2501.14733* (2024).

[18] Marius Mosbach, Tiago Pimentel, Shauli Ravfogel, Dietrich Klakow, and Yanai Elazar. 2023. Few-shot Fine-tuning vs. In-context Learning: A Fair Comparison and Evaluation. arXiv:2305.16938 [cs.CL] https://arxiv.org/abs/2305.16938

[19] Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. 2025. Nomic Embed: Training a Reproducible Long Context Text Embedder. arXiv:2402.01613 [cs.CL] https://arxiv.org/abs/2402.01613

[20] Long Ouyang and *et al.* 2022. Training language models to follow instructions with human feedback. arXiv:2203.02155 [cs.CL] https://arxiv.org/abs/2203.02155

[21] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830. https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html

[22] James C Phillips, Rosemary Braun, Wei Wang, James Gumbart, Emad Tajkhorshid, Elizabeth Villa, Christophe Chipot, Robert D Skeel, Laxmikant Kale, and Klaus Schulten. 2005. Scalable molecular dynamics with NAMD. *Journal of Computational Chemistry* 26, 16 (2005), 1781–1802.

[23] Alec Radford and Karthik Narasimhan. 2018. Improving Language Understanding by Generative Pre-Training. https://api.semanticscholar.org/CorpusID:49313245

[24] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. arXiv:1910.02054 [cs.LG] https://arxiv.org/abs/1910.02054

[25] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale Model Training. arXiv:2101.06840 [cs.DC] https://arxiv.org/abs/2101.06840

[26] William C Skamarock, Joseph B Klemp, and Jimy Dudhia. 2001. Prototypes for the WRF (Weather Research and Forecasting) model. In *Preprints, Ninth Conf. Mesoscale Processes*. Amer. Meteorol. Soc., J11–J15.

[27] Heydar Soudani, Evangelos Kanoulas, and Faegheh Hasibi. 2024. Fine Tuning vs. Retrieval Augmented Generation for Less Popular Knowledge. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region (SIGIR-AP 2024)*. ACM, 12–22. doi:10.1145/3673791.3698415

[28] Tianhua Tao, Junbo Li, Bowen Tan, Hongyi Wang, William Marshall, Bhargav M Kanakiya, Joel Hestness, Natalia Vassilieva, Zhiqiang Shen, Eric P. Xing, and Zhengzhong Liu. 2024. Crystal: Illuminating LLM Abilities on Language and Code. arXiv:2411.04156 [cs.SE] https://arxiv.org/abs/2411.04156

[29] Adina Williams, Nikita Nangia, and Samuel Bowman. 2018. A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, Marilyn Walker, Heng Ji, and Amanda Stent (Eds.). Association for Computational Linguistics, New Orleans, Louisiana, 1112–1122. doi:10.18653/v1/N18-1101

[30] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, Zhongzhu Zhou, Michael Wyatt, Molly Smith, Lev Kurilenko, Heyang Qin, Masahiro Tanaka, Shuai Che, Shuaiwen Leon Song, and Yuxiong He. 2023. DeepSpeed-Chat: Easy, Fast and Affordable RLHF Training of ChatGPT-like Models at All Scales. arXiv:2308.01320 [cs.LG]

[31] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. arXiv:2205.01068 [cs.CL]

[32] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv:2306.05685 [cs.CL] https://arxiv.org/abs/2306.05685

[33] Chunting Zhou, Pengfei Liu, Puxin Xu, Srini Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, Susan Zhang, Gargi Ghosh, Mike Lewis, Luke Zettlemoyer, and Omer Levy. 2023. LIMA: Less Is More for Alignment. arXiv:2305.11206 [cs.CL]